

AI 기반 인프라·앱·데이터 활용 기초

2주차 · 강의 및 실습 자료

메타넷디지털 서경진

주관 교육부 Ministry of Education | 국가평생교육진흥원

주차 개요 — 3개 주제

이번 주차에서 다룰 3개 핵심 주제를 한눈에 살펴봅니다.

1 AI 기반 인프라

- 분석·설계·구축
- IaC·MLOps 기초
- 확장성/관측성

2 AI 기반 애플리케이션

- 분석·설계·개발·테스트
- SDD·AI 통합 패턴
- 품질/평가·검증

3 AI 기반 데이터

- 분석·설계·개발·검증
- 데이터 파이프라인
- 품질·거버넌스

01

2주차 | SECTION

AI 기반 인프라 활용 기초

분석 → 설계 → 구축. AI 워크로드를 받치는 인프라를 설계한다.

학습 목표

AI 인프라·애플리케이션·데이터와 명세 우선(SDD) 개발의 토대를 익히고 Claude Code로 실습합니다.

- AI 인프라(컴퓨팅·스토리지·네트워크)를 IaC로 선언하고 MLOps 수명주기로 재현 가능하게 운영한다
- AI 앱 라이프사이클과 명세 우선(SDD)·검증 게이트로 "맞게 작동하는" 애플리케이션을 만든다
- 데이터 파이프라인(수집→정제→변환→적재)과 품질 5차원으로 신뢰할 수 있는 데이터를 다룬다
- EARS 5패턴으로 검증 가능한 요구사항·수용기준(AC)을 작성하고 테스트와 1:1로 매핑한다
- Claude Code 프롬프트와 명령(PowerShell·Bash)으로 인프라·AC·데이터 검증·SDD를 직접 실습한다

AI 엔지니어링 5대 개념

엔지니어링이 "좋은 질문 던지기"에서 "스스로 사고·행동하는 시스템 설계"로 확장됩니다.

1 프롬프트

- 입력(프롬프트) 구조화·설계
- Role·Few-shot·CoT
- 제로샷 추론 극대화

2 컨텍스트

- 외부 맥락·데이터 주입
- RAG·VectorDB·청킹·MCP
- 컨텍스트 창 효율화

3 하네스

- 품질·안전성 평가 환경
- LLM-as-judge·벤치마크
- 환각·성능저하 방지

4 루프

- 생성→피드백→수정 반복
- Self-Reflection·Critic-Actor
- 결과물 지속 개선

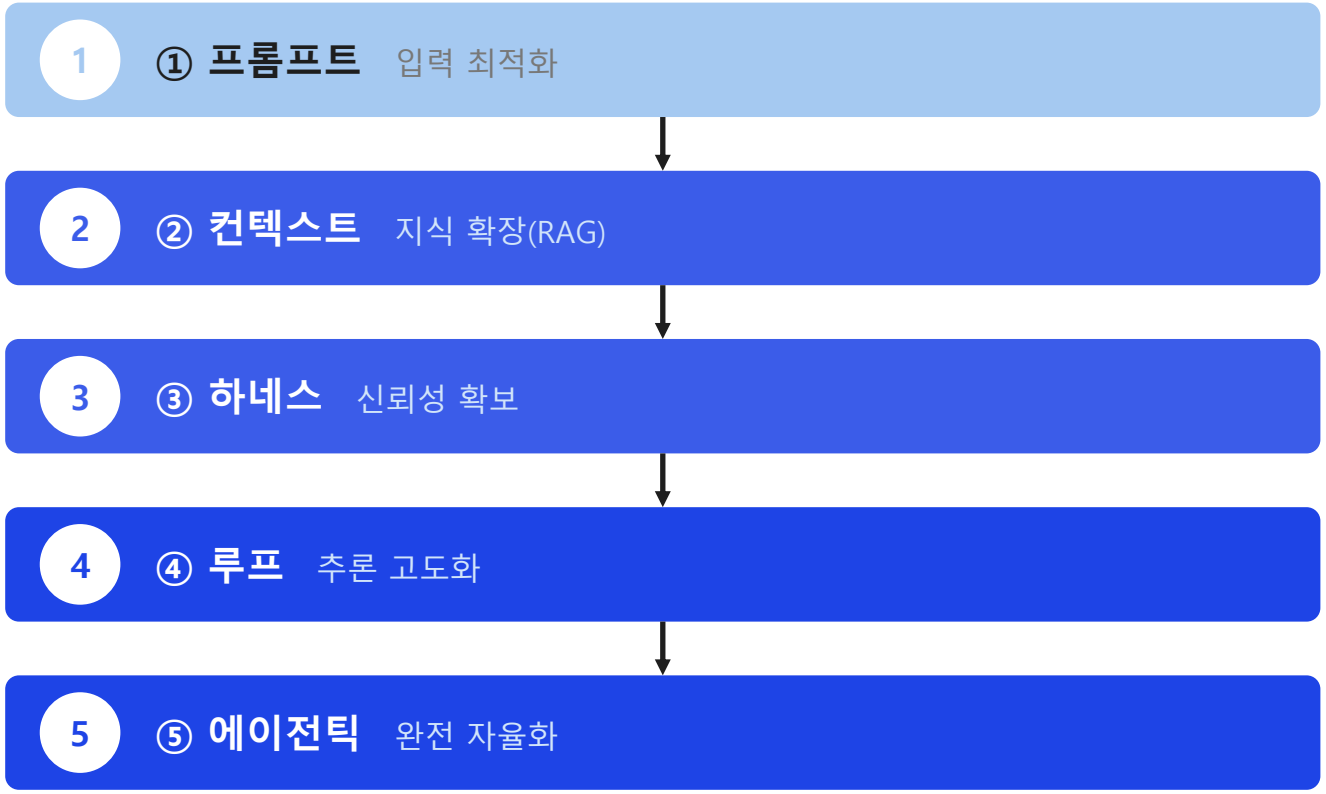
5 에이전틱

- 자율 계획·도구·실행
- LangGraph·n8n·Flowise
- 멀티에이전트 자율 완수

출처 LLM 시스템 엔지니어링 5단계(사용자 제공 자료)

AI 엔지니어링 진화 5단계 (Evolution Path)

각 단계는 독립 기술이 아니라 앞 단계 위에 누적·확장되는 진화 단계입니다.



핵심 설명

1. **1단계 · 프롬프트**
   1:1 대화로 입력 최적화 — 정적 지식의 한계를 인지
2. **2단계 · 컨텍스트**
   RAG로 외부 데이터 결합 — 검증 필요성이 대두
3. **3단계 · 하네스**
   자동 평가·테스트 파이프라인 — 단일 요청의 한계 인지
4. **4단계 · 루프**
   자기 수정 반복문 — 다중 작업·도구 제어 필요
5. **5단계 · 에이전틱**
   모든 단계를 조립 → 목표만 주면 자율적으로 완수

출처 LLM 시스템 엔지니어링 5단계(사용자 제공 자료)

한 줄 요약 — AI를 "독립 에이전트"로 키우는 과정

프롬프트→컨텍스트→하네스→루프→에이전틱으로 시스템이 단계적으로 자율화됩니다.

- 프롬프트 — 말귀를 알아듣게 만들고
- 컨텍스트 — 참고할 책(지식)을 쥐여준 뒤
- 하네스 — 시험을 쳐서 일을 잘하는지 감시하고
- 루프 — 틀린 것을 스스로 고치게 훈련시켜
- 에이전틱 — 마침내 혼자서도 업무를 완수하는 에이전트로 독립시킨다

출처 *LLM 시스템 엔지니어링 5단계(사용자 제공 자료)*

AI 인프라 구성 요소

AI 인프라는 컴퓨팅·스토리지·네트워크·관측성으로 구성됩니다.

1 컴퓨팅

- GPU/CPU, 오토스케일
- 서버리스(Functions)

2 스토리지

- 오브젝트·파일
- 벡터·캐시 저장소

3 네트워크/보안

- 프라이빗 엔드포인트
- ID·키 관리

4 관측성

- 로그·메트릭·트레이스
- 비용 모니터링

출처 *Amershi et al., "Software Engineering for ML"(ICSE 2019) · Sculley et al., "Hidden Technical Debt in ML"(NeurIPS 2015)*

AI 인프라 아키텍처

AI 워크로드는 컴퓨팅·스토리지·네트워크 위에서 관측성으로 운영됩니다.



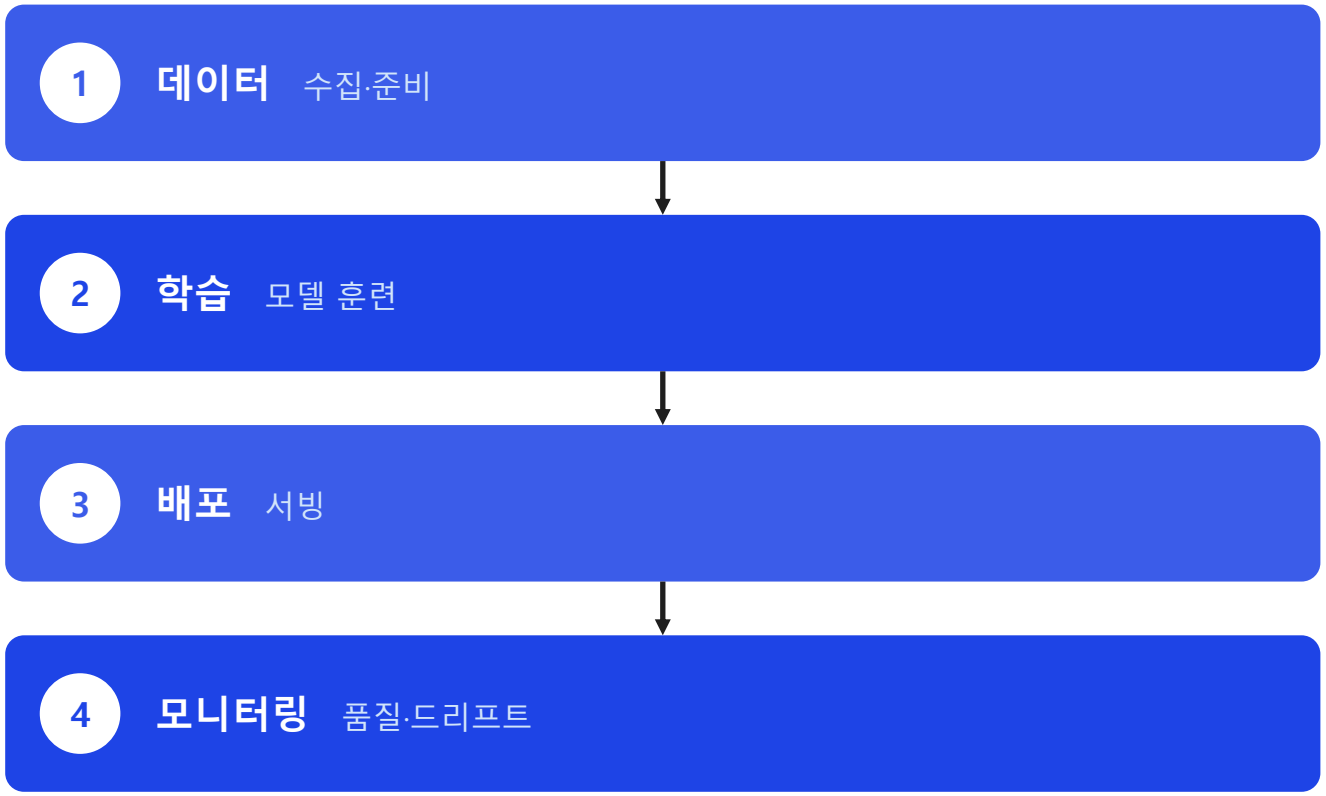
핵심 설명

1. **관측성**
   로그·메트릭·트레이스로 운영 가시화
2. **컴퓨팅**
   GPU/CPU·오토스케일·서버리스
3. **스토리지**
   오브젝트·벡터 DB·캐시
4. **네트워크/보안**
   프라이빗 엔드포인트·ID/키 관리
5. **IaC**
   코드로 선언해 재현 가능하게 구축

출처 *MLOps: Overview, Definition, and Architecture(arXiv:2205.02302)*

MLOps 수명주기

배포 후 모니터링 결과로 다시 데이터·학습을 개선하는 순환 구조입니다.



핵심 설명

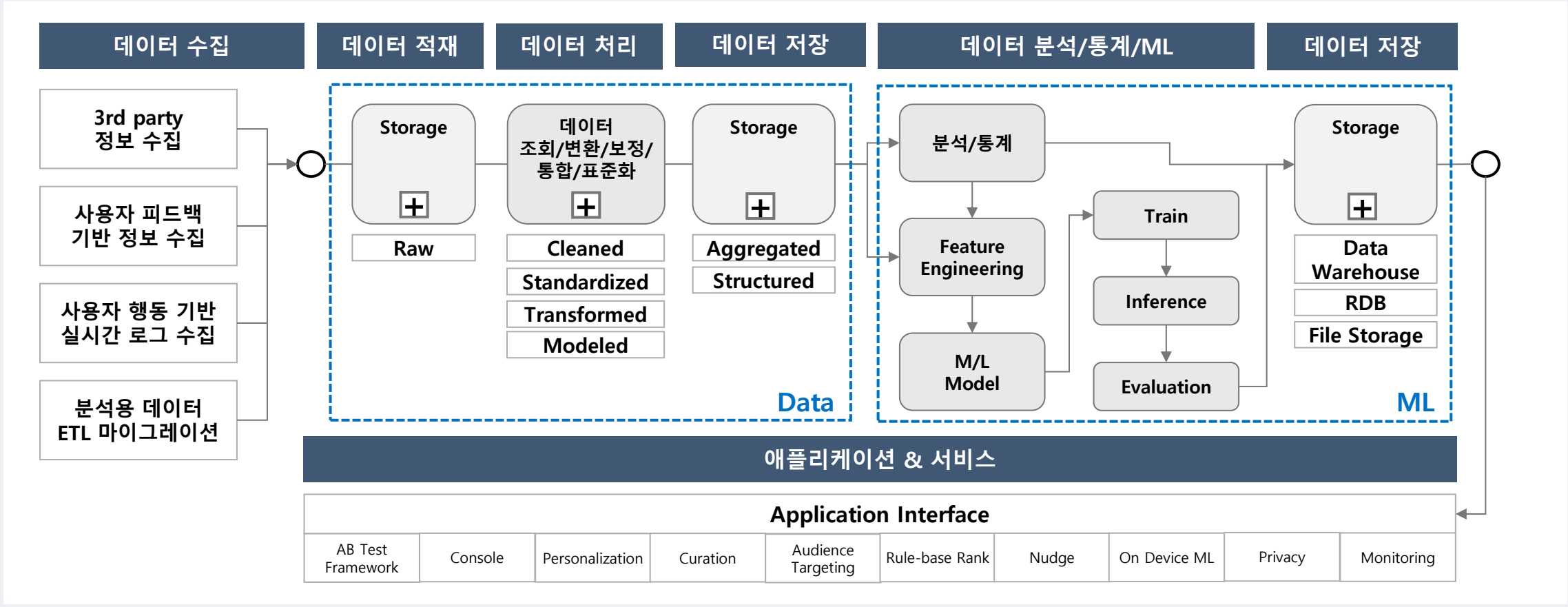
1. **데이터**
   수집·정제·피처 준비
2. **학습**
   모델 훈련·실험 관리
3. **배포**
   서빙·버전 관리
4. **모니터링**
   품질·드리프트 감시
5. **지속 학습**
   모니터링 결과로 재학습 순환

출처 MLOps Survey(2205.02302) · 패스트캠퍼스 "MLOps 개념"(YouTube)

DataOps/MLOps 구축 사례

AI Agent 서비스 제공을 위한 모델 학습 DataOps/MLOps 구축 사례입니다.

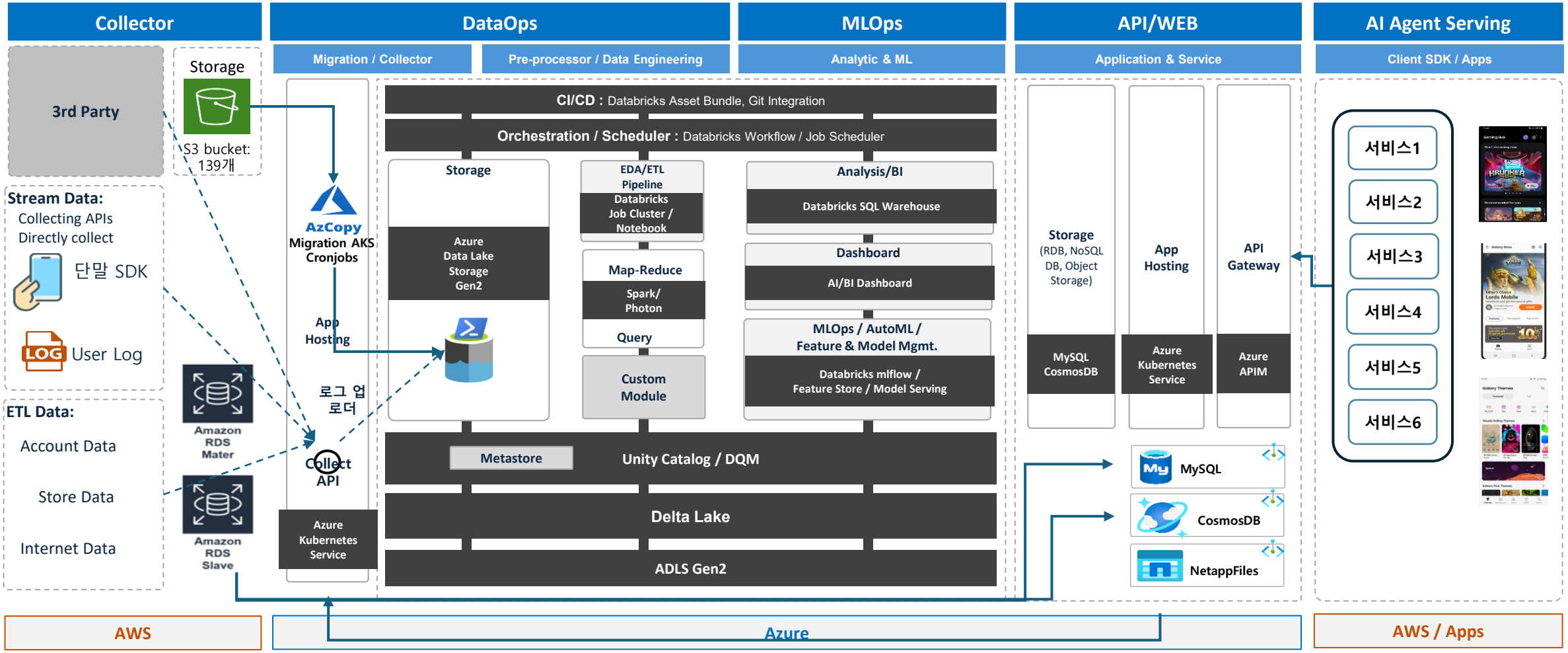
예시 DataOps/MLOps 기반 AI Agent 서비스 구성도



출처 *사내 DataOps/MLOps 구축 사례*

AI Agent 서비스 구축 사례

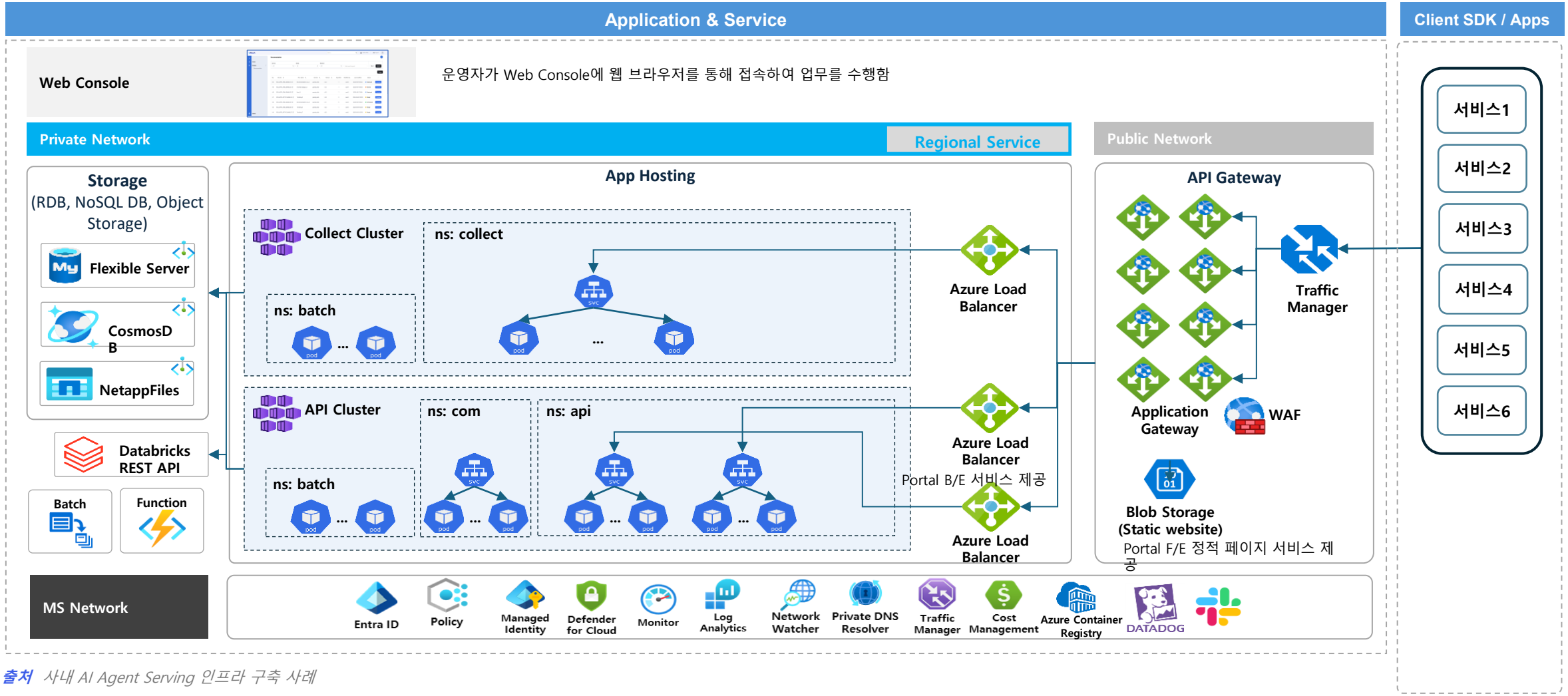
Azure Public Cloud 기반 AI 모델 학습 및 AI Agent 서비스 구축 사례입니다.



출처 *사내 Azure·Databricks 기반 AI Agent 서비스 구축 사례*

AI Agent 인프라 구축 사례

DataOps/MLOps 기반 AI Agent Serving 인프라 구축 사례입니다.



출처 사내 AI Agent Serving 인프라 구축 사례

2주차 실습 구성 — SDD·IaC·데이터

5개 HANDS-ON을 3개 통합 실습으로 재구성 · 상세: prompt/2주차_실습_프롬프트.md

구성

SDD로 클라우드·로컬 3-tier + 데이터 품질 게이트 · 산출물: 실습산출물/2주차/

통합 실습 목록

2-1 SDD→클라우드·로컬 3-tier→데이터 품질 게이트

2-2 EARS 수용기준→테스트 1:1 매핑

2-3 2주차 실습 요약 문서 생성

학습 흐름

```
명세(spec) 먼저 (SDD)
    ↓
클라우드 IaC (Azure/Terraform)
    ↓
로컬 배포 (WSL2 + Kubernetes)
    ↓
데이터 품질 게이트 (pandas)
    ↓
AC↔구현 매핑 리포트
```

실습 2-1 · SDD 3-tier 데이터 플랫폼

동일 명세로 클라우드·로컬 배포 + 데이터 품질 검증

실습 목표 명세→클라우드·로컬 3-tier→데이터 품질 게이트를 하나의 흐름으로

진행 단계

1. spec에 EARS AC(클라우드·로컬·데이터)
2. Azure 3-tier Terraform(infra/) validate
3. 로컬 kind 3-tier(k8s/) 적용
4. 데이터 품질 validate.py(게이트) 실행

프롬프트

```
'고객 이벤트 3-tier 데이터 플랫폼'을 SDD로.
동일 설계를 Azure(App Service/VMSS/SQL)와
로컬 K8s(nginx/node/postgres)에 배포하고,
이벤트 CSV 품질(결측·중복·이상치, 임계95%)을
게이트로 검증. 산출물: 실습산출물/2주차/
```

02

2주차 | SECTION

AI 기반 애플리케이션 활용 기초

분석 → 설계 → 개발 → 테스트. AI를 품은 앱의 라이프사이클.

AI 앱 개발 라이프사이클



출처 AI 앱 개발 라이프사이클(SDLC) · 2026 매치업 X+AI 교육과정

Spec-Driven Development(SDD) 기초

"돌아간다 ≠ 맞다" — 명세를 먼저 쓰고 코드가 그 명세를 따르게 합니다.

Vibe coding (명세 없음)

- "일단 만들어줘" 즉흥 생성
- happy path만 동작
  - 빠진 요구사항이 조용히 배포
- "돌아간다 ≠ 맞다"

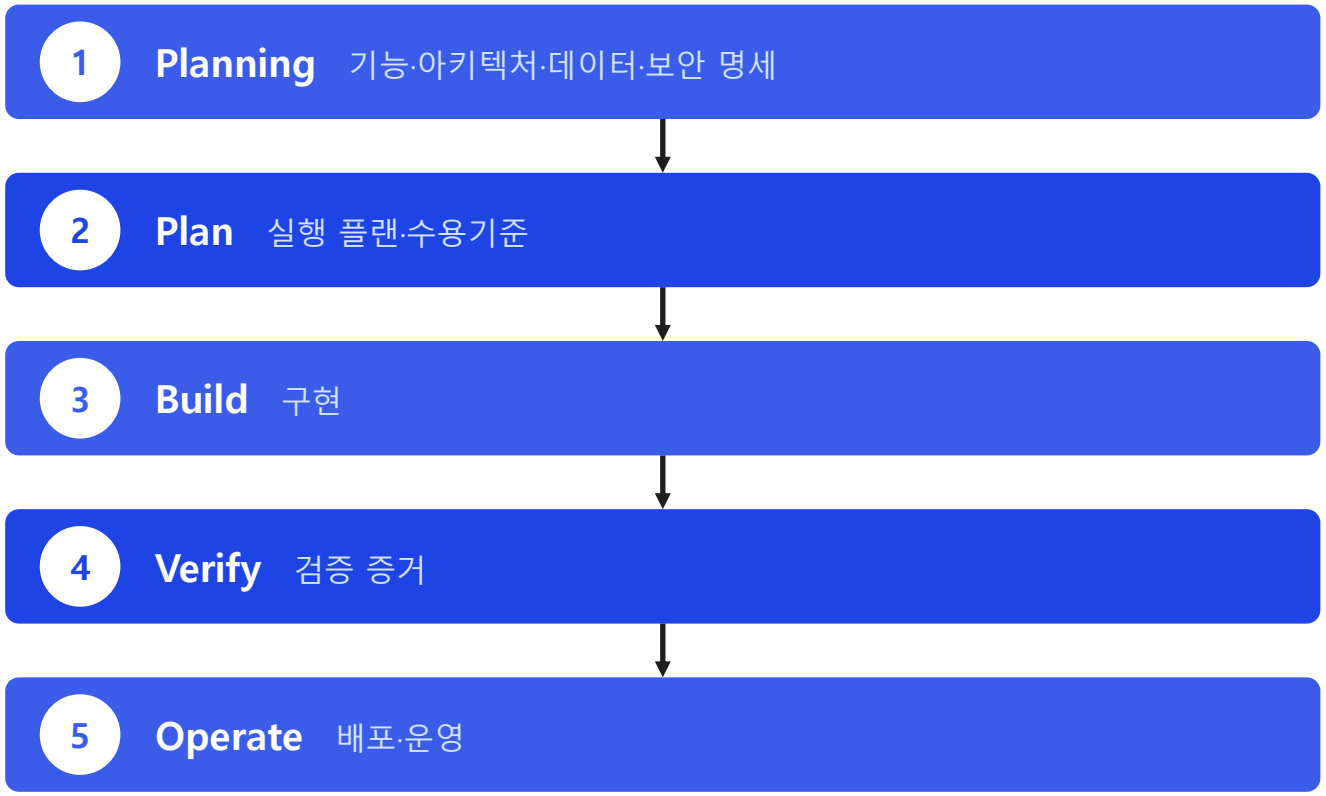
SDD (명세 먼저)

- 검증 가능한 명세를 먼저 작성
- 명세 3요소: 의도·수용기준·제약
  - 수용기준(AC) ↔ 테스트 1:1
- 명세가 단일 진실(SSOT)

출처 *에이전틱 코딩 & SDD 심화(사내) · "Requirement Engineering in AI Native SDLC"(MAIPD, 사내)*

SDD 5단계 파이프라인

명세→플랜→구현→검증→운영. 각 단계 산출물을 표준 폴더에 남깁니다(00_sources~05_operate).



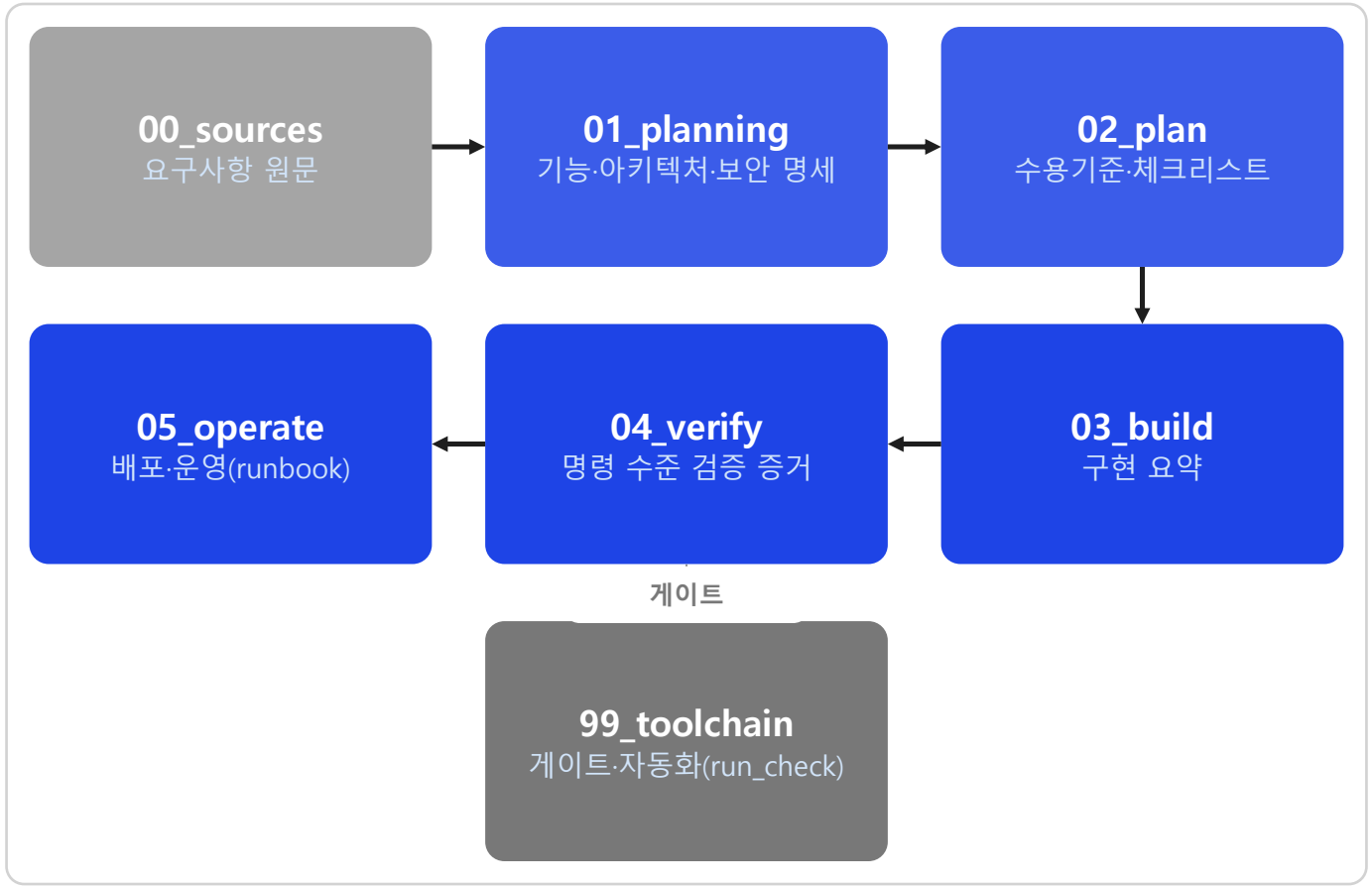
핵심 설명

1. **Planning**
   기능·아키텍처·데이터·보안 명세 작성
2. **Plan**
   수용기준·체크리스트로 실행 계획
3. **Build**
   명세를 따라 구현
4. **Verify**
   명령 수준 검증 증거 확보
5. **Operate**
   배포·운영 — 드리프트 시 명세 우선 수정

출처 에이전틱 코딩 & SDD 심화(사내) · SDD 하네스 분석(agentic-dev)

SDD 디렉터리 = SDLC 게이트

"어디에 무엇을 남길지"가 규약이며, 게이트가 완료 기준을 기계로 강제합니다.



핵심 설명

1. **01_planning**
   기능·아키텍처·보안 명세를 코딩 전에 작성
2. **02_plan**
   수용기준·체크리스트로 실행 계획 수립
3. **03_build**
   구현 요약을 사실 기반으로 기록
4. **04_verify**
   명령 수준 검증 증거를 남김
5. **99_toolchain**
   게이트가 완료 기준을 기계로 강제

출처 SDD 하네스 분석(agentic-dev, 사내)

검증 게이트 — 완료의 기준

완료는 "느낌"이 아니라 게이트(테스트·체크) 통과로 정의합니다.

사람이 봤다

- "다 됐습니다" 선언
- happy path만 확인
- 회귀 누락 위험
- 주관적 판단

증거 기반 완료

게이트가 통과시켰다

- 명령 수준 증거
- 회귀 범위까지 검증
- 재현 가능(결정적)
- exit 0 = 완료

출처 *SDD 하네스 분석(사내) · Harness Engineering 세미나(사내)*

AI 통합 패턴과 검증

AI를 앱에 통합하는 패턴과, 증거 기반으로 품질을 검증하는 관점을 익힙니다.

AI 통합 패턴

- 프롬프트 기반 호출
- RAG로 지식 주입
  - 함수 호출/도구 연동
- 에이전트 오케스트레이션

검증 관점

- 기능·예외·환각 점검
- 출력 품질 평가(LLM-as-judge)
  - 증거 기반 완료(검증 게이트)
- 회귀 테스트셋 관리

출처 Anthropic "Building Effective Agents"(2024) · SDD 하네스 분석(사내)

03

2주차 | SECTION

AI 기반 데이터 활용 기초

분석 → 설계 → 개발 → 검증. AI의 연료, 데이터를 다룬다.

데이터 파이프라인 기초

"Garbage in, garbage out" — 데이터 품질이 AI 품질을 결정합니다.

- 수집(Ingest) → 정제(Clean) → 변환(Transform) → 적재(Load)
  - 배치 vs 스트리밍 처리의 차이
- 데이터 품질 차원: 정확성·완전성·일관성·최신성
- ML/AI용 피처·임베딩 준비도 파이프라인에 포함(학습/서빙 일관성)
- 거버넌스: 개인정보·접근 권한·데이터 계보(lineage)

출처 MLOps: Overview, Definition, and Architecture(arXiv:2205.02302) · 패스트캠퍼스 "백엔드 개발자를 위한 MLOps"(YouTube)

데이터 엔지니어링 파이프라인

원천 데이터를 정제·변환하여 AI가 쓸 수 있는 피처·임베딩으로 만듭니다.



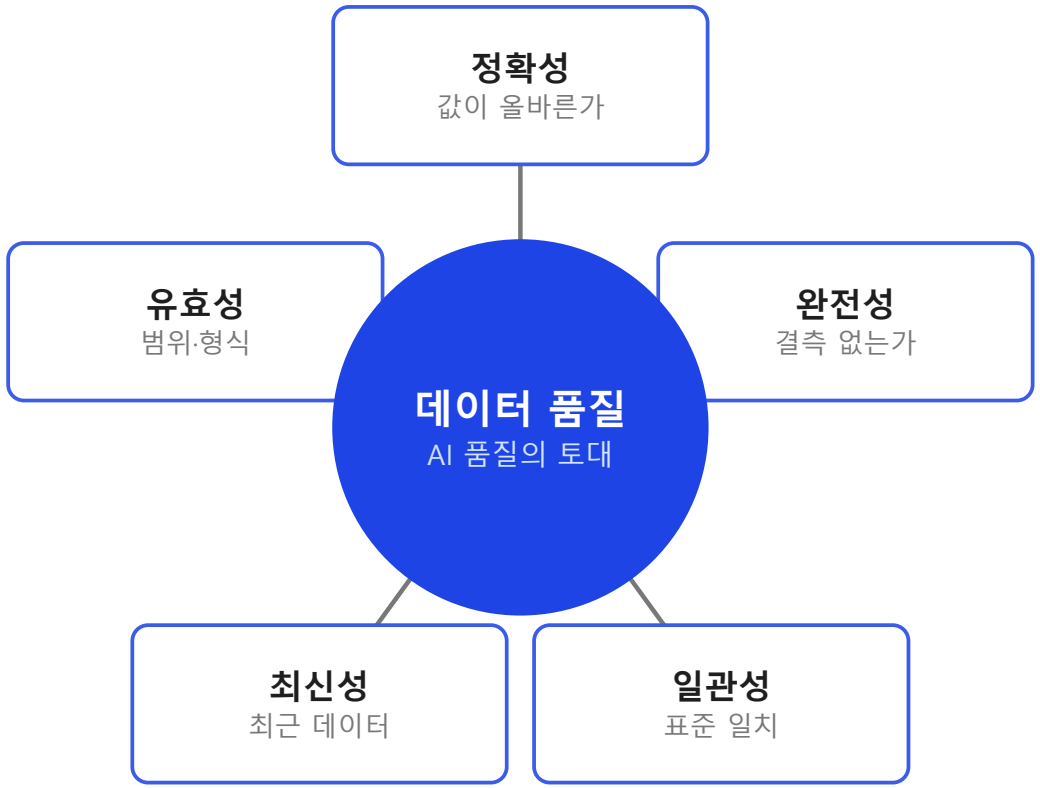
핵심 설명

1. **수집**
   다양한 소스에서 원천 데이터 확보
2. **정제**
   결측·중복·노이즈 제거
3. **변환**
   표준 포맷·스키마로 가공
4. **적재**
   저장소/인덱스에 로드
5. **피처/임베딩**
   AI가 쓸 수 있는 형태로 준비

출처 *MLOps Survey(2205.02302)*

데이터 품질 5차원

"Garbage in, garbage out" — 품질 차원으로 데이터를 점검합니다.



핵심 설명

1. **정확성**
   값이 사실과 일치하는가
2. **완전성**
   결측 없이 채워졌는가
3. **일관성**
   단위·코드가 표준화됐는가
4. **최신성**
   최근 데이터를 반영하는가
5. **유효성**
   허용 범위·형식을 지키는가

출처 *데이터 검증 체크리스트 · MLOps Survey(2205.02302)*

데이터 검증 체크리스트

스키마·범위·중복·일관성 기준으로 데이터 품질을 점검합니다.

항목	점검 내용	예시
스키마	필드 타입·필수값	null/형식 오류
범위	값의 유효 범위	음수 나이
중복	중복 레코드 제거	동일 ID
일관성	단위·코드 표준화	날짜 포맷

04

2주차 | SECTION

SDD 심화 — 명세를 코드로

EARS 표기법과 검증 가능한 명세 작성을 익힌다.

SDD 계보 — 명세 먼저의 뿌리

"명세 먼저"는 수십 년 된 원리를 AI 시대에 재발견한 것입니다.

시대	접근	핵심 아이디어
1970s	Z · VDM	수학적 명세로 코드 전에 시스템을 기술
1999~	TDD	테스트를 코드보다 먼저 = 실행 가능한 명세
2000s	BDD	자연어 Given/When/Then으로 행위 명세
2009	EARS	요구사항을 쓰는 경량 정형 문법
2025~	SDD 도구	AI 실행 주체에 맞춰 재발견

출처 SDD 심화 — 계보(Z·VDM·TDD·BDD·EARS, 사내)

모호한 요구 vs EARS 문장

"안전하게"처럼 모호한 문장은 테스트도 AI 파싱도 불가능합니다.

모호 (안티패턴)

- "OTP를 안전하게 처리"
- 트리거 불명확(언제?)
- 측정 불가(몇 초/몇 회?)
- 테스트·AI 파싱 불가

정형화

EARS (검증 가능)

- When 조건을 명시
- shall 동작을 명시
- 임계값 명시(예: 300초)
- 테스트 1개로 직결

출처 planning — EARS(사내)

EARS 5패턴

EARS는 요구를 5가지 정형 문장(shall)으로 쓰는 경량 문법입니다.

1 Ubiquitous

- 상시 불변 규칙
- The system shall …

2 Event-driven

- When 트리거
- When …, shall …

3 State-driven

- While 상태
- While …, shall …

4 Optional

- Where 기능 플래그
- Where …, shall …

5 Unwanted

- If/Then 금지
- If …, shall not …

출처 *planning — EARS 5패턴(Mavin, 2009)*

SDD 효용 6가지

명세가 닻이 되어 에이전틱 개발의 안전벨트 역할을 합니다.

1 drift 방지

- 명세가 닻 역할
- 의도 표류 차단

2 검증 가능

- 수용기준=합격/불합격
- 기계적 검증

3 안전한 재생성

- 회귀 범위 고정
- 재구현 위험↓

4 거버넌스 산출물

- 명세·검증=증빙
- 감사 대응

5 병렬 작업

- 겹치지 않는 명세
- 에이전트 분할

6 단일 진실

- spec = SSOT
- 변경 진입점 고정

출처 SDD 심화 — 효용 5가지(사내)

sdd/ 도메인 섹션 (정본 위치)

명세는 docs·위키 사본 금지 — sdd/ 정본에만 둡니다(Placement Rule).

기획·기능	01_feature	02_screen	
설계	03_architecture	04_data	05_api
연계·비기능	06_iac	07_integration	08_nonfunctional
보안·테스트	09_security	10_test	

출처 planning — section map(사내)

핵심 설명

1. **정본만 유지**
   같은 spec을 여러 곳에 두지 않음(drift 방지)
2. **Final-Only**
   날짜·버전 파일 대신 덮어쓰기
3. **진입점**
   INDEX.md가 1단 진입점
4. **10개 섹션**
   기능~테스트까지 표준 위치
5. **규약**
   "어디에 무엇을 남길지"는 토론 대상이 아님

2주차 핵심 정리

- AI 인프라(컴퓨팅·스토리지·네트워크)는 IaC로 선언하고 MLOps 수명주기로 배포·운영한다
- AI 앱은 명세 우선(SDD)·검증 게이트로 "돌아간다"를 넘어 "맞게" 만든다
- 데이터 파이프라인(수집→정제→변환→적재)과 품질 5차원이 AI 결과 품질을 좌우한다
- EARS 5패턴으로 검증 가능한 수용기준을 쓰고 테스트와 1:1로 완료를 증명한다
- Claude Code 프롬프트+명령(PowerShell·Bash)으로 인프라·AC·데이터 검증·SDD를 실습한다

▶ 다음 차시 3주차 — AI 개발 기초·데이터 엔지니어링·임베딩